METROPOLITAN TIRANA UNIVERSITY

Course: Data Structures and Algorithms

Building a Product Catalog System with Binary Search Tree

Evis Plaku

Why binary search trees?

METROPOLITAN TIRANA UNIVERSITY

Problem statement

Managing a large catalog of products

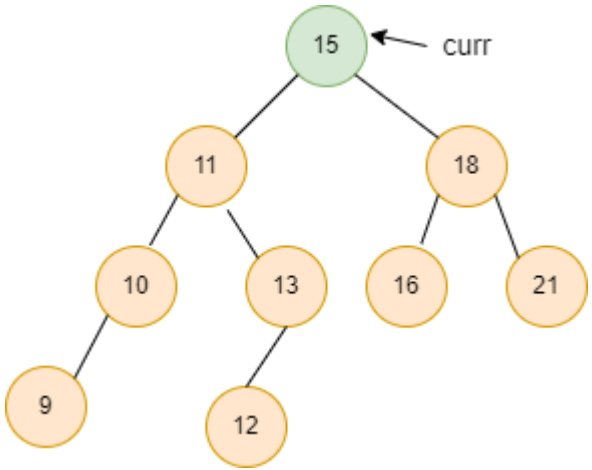
Problem introduction and key operations

Introduction to the problem

METROPOLITAN TIRANA UNIVERSITY

Efficiently manage and search large product catalogs with a Binary Search Tree

- **Scalability**: handle growing product data efficiently
- **Efficiency**: fast search and update operations
- **Real-World**: key for e-commerce and inventory management



Designing the Product Catalog System

METROPOLITAN TIRANA UNIVERSITY

Product

- Define `Product` class
- `id`, `name` and `price` as attributes
- Method to `compare` products

BST

- BST representation of all products
- Methods to sort, search, insert, or remove
- All key functionalities related to BSTs

Main

- Load products from file
- Menu-driven application for key functionalities
- Customized

Data model: the Product class

Represents individual products with attributes for name, price, and ID

- Attributes: name, price, and unique product ID
- Comparable: enables sorting by product ID
- Utility: used in managing product catalog in BST

```java
public class Product implements Comparable<Product> {
    private String name;
    private int id;
    private double price;

    @Override
    public int compareTo(Product other) {
        return Integer.compare(this.id, other.id);
    }

}
```

BST operations: insert product

Products are inserted based on chosen sorting criteria (name or price)

- Traverse the tree, compare with node value, and insert at the correct position

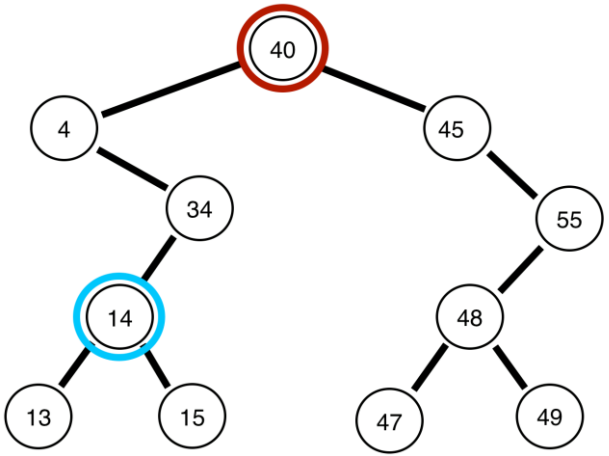
```
public void insert(T value) {
        root = insertRecursive(root, value);
}

private Node<T> insertRecursive(Node<T> current, T value) {
        if (current == null) return new Node<>(value);

        if (compare(value, current.data) < 0) {
            current.left = insertRecursive(current.left, value);
        } else if (compare(value, current.data) > 0) {
            current.right = insertRecursive(current.right, value);
        }
        retun current;
}
```

BST operations: search for product

METROPOLITAN TIRANA UNIVERSITY

Products are inserted based on chosen sorting criteria (name or price)

- Traverse the tree, comparing the product's attribute (name or price) to each node




```java
public boolean contains(T value) {
        return containsRecursive(root, value);
}

private boolean containsRecursive(Node<T> current, T value) {
        if (current == null) return false;

        int cmp = compare(value, current.data);
        if (cmp == 0) return true;
        if (cmp < 0) return containsRecursive(current.left, value);
        else return containsRecursive(current.right, value);
}
```

BST operations: search for product

METROPOLITAN TIRANA UNIVERSITY

Find the product in the tree and remove it, adjusting the tree structure if necessary

- If the node has no children, remove it directly
- If it has one child, link its parent to the child
- If it has two children, replace it with the smallest node from the right subtree (in-order successor)

```java
private Node<T> deleteRecursive(Node<T> current, T value) {
        if (current == null) return null;

        int cmp = compare(value, current.data);
        if (cmp < 0) {
            current.left = deleteRecursive(current.left, value);
        } else if (cmp > 0) {
            current.right = deleteRecursive(current.right, value);
        } else {
            if (current.left == null) return current.right;
            if (current.right == null) return current.left;

            T smallest = findMin(current.right);
            current.data = smallest;
            current.right = deleteRecursive(current.right, smallest);
        }
        return current;
}
```

Analyzing the tree structure

Tree stats: analyzing the tree structure

METROPOLITAN TIRANA UNIVERSITY

- **Total nodes**: count all nodes in the tree
- **Leaf nodes**: nodes with no children
- **Internal nodes**: nodes with at least one child
- **Height**: maximum depth from the root to a leaf
- **Balance check**: determine if the tree is balanced

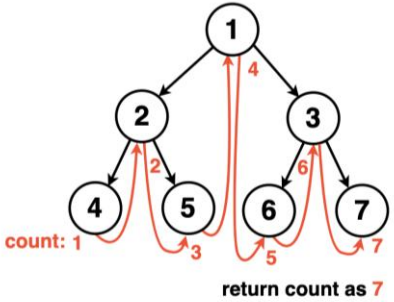
Tree stats reveal efficiency, balance, and structure for optimal performance of the tree

Tree stats: analyzing the tree structure

METROPOLITAN TIRANA UNIVERSITY

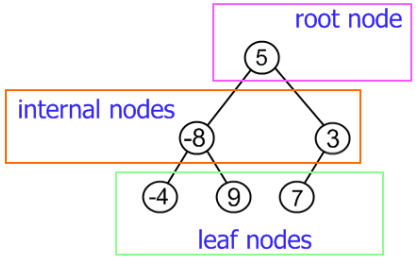
Total nodes

1. Start from the root
2. Recursively traverse all nodes
3. Increment count for each node visited
4. Return the total count



Leaf nodes

1. Start from the root
2. Recursively traverse all nodes
3. Check if node has no children
4. Increment leaf count for nodes with no children
5. Return total leaf count



Internal nodes

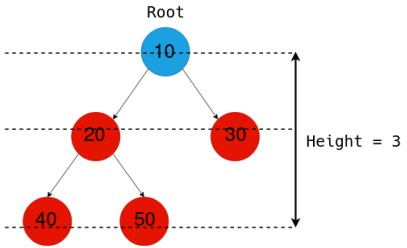
1. Start from the root
2. Recursively traverse all nodes
3. Check if node has at least one child
4. Increment the total count for such nodes
5. Return total count of internal nodes

Tree stats: analyzing the tree structure

METROPOLITAN TIRANA UNIVERSITY

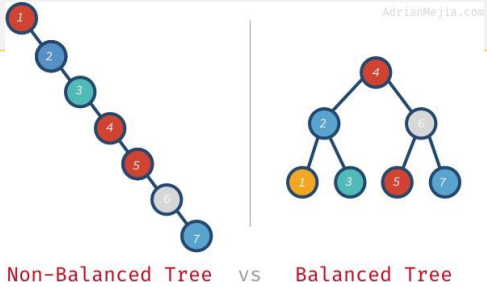
Height

1. Start from the root
2. Recursively calculate the depth of the left and right subtrees
3. Return the maximum depth of the subtrees plus 1 for the current node
4. The height of the tree is the maximum depth from root to leaf



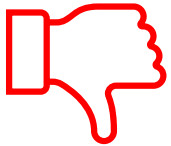
Balance check

1. Start from the root
2. Recursively calculate the depth of the left and right subtrees
3. Check if the difference in height between the subtrees is more than 1
4. If any node has a difference greater than 1, the tree is unbalanced
5. If no imbalance is found, return that the tree is balanced



Quote of the Week

METROPOLITAN TIRANA UNIVERSITY

In a world of linear thinkers

be like a Binary Search Tree